Math appendix

Every loss, gradient and metric used in SpaceGen AI, derived step by step. Each section ends with where the result is used in the code and what checks it. Section 11 lists the checks in one table.

Notation: B is the batch size, $K = 6$ the number of furniture slots, $W \times D$ the room in meters. A layout places item k at centre (x_k, y_k) with rotation class $r_k \in \{0, 1, 2, 3\}$. The mask $m_k \in \{0, 1\}$ says whether item k is present. $\odot$ is the element-wise product.

---

1. Shapes, scaling and masking

Forward pass of one layer. For a batch $X \in \mathbb{R}^{B \times d_{\text{in}}}$, weights $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and bias $b \in \mathbb{R}^{d_{\text{out}}}$:

$$Z = XW + b, \qquad H = \operatorname{act}(Z), \qquad Z, H \in \mathbb{R}^{B \times d_{\text{out}}}.$$

The product XW exists only if the number of columns of X equals the number of rows of W. The bias is added to every row (broadcasting).

Scaling. Positions are stored as fractions of the room: $u_k = x_k / W$, $v_k = y_k / D$, both in $[0, 1]$. This is a scalar multiplication per axis. It makes the target independent of the room's absolute size and matches the range of the Sigmoid output.

Masking. The loss of item k is multiplied by m_k. An absent item ($m_k = 0$) therefore contributes zero loss and, because $\partial(m_k \ell_k)/\partial\theta = m_k \, \partial\ell_k/\partial\theta$, zero gradient.

Used in: `spacegen/dataset.py` (encoding), `spacegen/models/cvae.py` (`cvae_loss`). *Checked by:* the round-trip test of the encoding and the masking test of the loss.

---

2. Output activations and their losses

2.1 Sigmoid with binary cross-entropy (the evaluator's validity output)

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\,(1 - \sigma(z)) \le \tfrac{1}{4}.$$

With $p = \sigma(z)$ and a label $y \in \{0, 1\}$, the loss is $L = -\left[\, y \log p + (1 - y) \log(1 - p) \,\right]$. Then

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p} = \frac{p - y}{p\,(1 - p)}, \qquad \frac{\partial p}{\partial z} = p\,(1 - p) \quad \Longrightarrow \quad \frac{\partial L}{\partial z} = p - y.$$

The factor $p(1 - p)$ cancels, so the gradient does not vanish when the prediction is confidently wrong. That is why BCE is paired with Sigmoid, and why the code uses the logit form (`binary_cross_entropy_with_logits`), which never computes $\log 0$.

2.2 Softmax with categorical cross-entropy (the rotation head)

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad \frac{\partial p_i}{\partial z_k} = p_i\,(\delta_{ik} - p_k), \qquad L = -\sum_i y_i \log p_i.$$

$$\frac{\partial L}{\partial z_k} = -\sum_i \frac{y_i}{p_i}\, p_i\,(\delta_{ik} - p_k) = -y_k + p_k \sum_i y_i = p_k - y_k,$$

because the one-hot label sums to 1. The network outputs the logits z; `F.cross_entropy` applies the softmax inside the loss.

2.3 MSE, MAE and Huber (the position head)

For one coordinate with error $e = \hat{y} - y$:

Loss	Value	Gradient $\partial L/\partial e$	Behaviour
MSE	e^2	$2e$	grows with the error, so large errors (outliers) dominate
MAE	$\lvert e\rvert$	$\operatorname{sign}(e)$	constant size; not differentiable at 0 (any value in $[-1, 1]$ is a subgradient)
Huber, threshold δ	$\frac{1}{2}e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \frac{1}{2}\delta)$	e or $\delta \operatorname{sign}(e)$	quadratic near 0, linear beyond δ

In the code the position loss of an item is the sum over its two coordinates, multiplied by the mask and averaged over the batch.

The loss scale sets the effective KL weight. The CVAE minimizes reconstruction $+ \beta \cdot$ KL (Section 6). For a typical error of $\lvert e\rvert = 0.1$ in normalized coordinates, MSE gives 0.01 and MAE gives 0.1. With the same β, MAE's reconstruction term weighs ten times more against the KL term, so the encoder keeps more information in z. Huber with $\delta = 0.01$ gives about $\delta\lvert e\rvert = 0.001$: its gradients are tiny, and the KL term wins. Experiment E2 shows exactly this order: mean position error 0.19 m for MAE, 0.58 m for MSE, 0.84 to 0.90 m for Huber.

In probabilistic terms, MSE is the negative log-likelihood of a Gaussian noise model and MAE that of a Laplace noise model; the noise scale plays the role of β.

Used in: `position_error` in `spacegen/models/cvae.py`, `evaluator_loss` in `spacegen/models/evaluator.py`. *Checked by:* the hand-derived gradients of `notebooks/01_manual_backprop.ipynb`, which equal autograd to 1.8×10^{-15}. *Decided by:* E2 and the Gate 2 comparison: MAE is the frozen position loss.

---

3. Hidden activations and gradient flow

Activation	$\operatorname{act}(z)$	$\operatorname{act}'(z)$
Sigmoid	$\sigma(z)$	$\sigma(z)(1-\sigma(z)) \le 0.25$
Tanh	$\tanh z$	$1 - \tanh^2 z \le 1$
ReLU	$\max(0, z)$	1 for $z > 0$, else 0 (0 is chosen at $z = 0$)
Leaky ReLU	z for $z > 0$, else αz	1 or α
ELU	z for $z > 0$, else $\alpha(e^z - 1)$	1 or αe^z
GELU	$z\,\Phi(z)$, with Φ the standard normal CDF	$\Phi(z) + z\,\phi(z)$

Vanishing gradients. Backpropagation multiplies by $\operatorname{act}'(z)$ at every layer (Section 4). Through n Sigmoid layers the factor is at most 0.25^n: for six layers, $0.25^6 \approx 2.4 \times 10^{-4}$, before the weights shrink it further. E3b measures a gradient norm of 10^{-10} at the encoder's input for Sigmoid at depth 6 without BatchNorm; the model stops learning (KL 0, no active latent unit).

Dying ReLU. A ReLU unit whose input is negative for every training sample has derivative 0 everywhere, so its weights never change again. E3b counts 29% such units at depth 6 without BatchNorm.

Why BatchNorm hides both. BatchNorm rescales each unit's pre-activations to about zero mean and unit variance over the batch. Sigmoid then works near $z = 0$, where σ' is at its maximum, and about half of every ReLU unit's inputs are positive. With BatchNorm on, E3b finds 0% dead ReLU units at depth 6 and a Sigmoid network that trains.

Used in: `ACTIVATIONS` in `spacegen/models/cvae.py`. *Checked by:* E3a and E3b (`reports/tables/e3a.csv`, `e3b.csv`).

---

4. Backpropagation through one layer

Let $Z = XW + b$ and $H = \operatorname{act}(Z)$, and suppose $G = \partial L/\partial H \in \mathbb{R}^{B \times d_{\text{out}}}$ is known. Define $\Delta = G \odot \operatorname{act}'(Z)$. Then

$$\frac{\partial L}{\partial W} = X^\top \Delta, \qquad \frac{\partial L}{\partial b} = \sum_{\text{batch}} \Delta, \qquad \frac{\partial L}{\partial X} = \Delta\, W^\top.$$

Why: $Z_{bj} = \sum_i X_{bi} W_{ij} + b_j$, so $\partial L / \partial W_{ij} = \sum_b X_{bi} \Delta_{bj}$, which is the (i, j) entry of $X^\top \Delta$. The other two follow the same way. $\partial L / \partial X$ is the G of the layer below, so the rule is applied layer after layer from the loss back to the input.

Checked by: `notebooks/01_manual_backprop.ipynb` derives these by hand for a two-item toy network (masked position loss, cross-entropy, penetration-depth overlap). The result equals autograd to 1.8×10^{-15} and finite differences to 1.8×10^{-9} (relative); `tests/test_notebooks.py` runs the notebook.

---

5. Optimizers

With gradient g_t at step t and learning rate η:

- **SGD:** $\theta \leftarrow \theta - \eta\, g_t$.
- **Momentum:** $v_t = \mu\, v_{t-1} + g_t$, $\theta \leftarrow \theta - \eta\, v_t$. Past gradients accumulate, which speeds up progress along directions the gradients agree on.
- **RMSProp:** $s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2$, $\theta \leftarrow \theta - \eta\, g_t / (\sqrt{s_t} + \epsilon)$. Each parameter's step is divided by a running size of its own gradient.
- **Adam:** both ideas, with bias correction:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta \leftarrow \theta - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Why the correction: $m_0 = 0$, so $\mathbb{E}[m_t] = (1-\beta_1^t)\, \mathbb{E}[g]$ for a steady gradient; dividing by $1-\beta_1^t$ removes the bias towards 0 in the first steps.

Used in: `make_optimizer` in `spacegen/train_cvae.py`. *Checked by:* E4: Adam at 10^{-3} reaches the target validation loss in 24 epochs, plain SGD at 0.01 in 153.

---

6. The conditional VAE

6.1 The evidence lower bound (ELBO)

The model is $p(x \mid c) = \int p(x \mid z, c)\, p(z)\, dz$ with prior $p(z) = \mathcal{N}(0, I)$. For any encoder distribution $q(z \mid x, c)$:

$$\log p(x \mid c) = \log \mathbb{E}_q\left[\frac{p(x \mid z, c)\, p(z)}{q(z \mid x, c)}\right] \;\geq\; \mathbb{E}_q\big[\log p(x \mid z, c)\big] - \mathrm{KL}\big(q(z \mid x, c) \,\|\, p(z)\big),$$

by Jensen's inequality ($\log$ is concave). The gap between the two sides is $\mathrm{KL}(q(z \mid x, c) \,\|\, p(z \mid x, c))$, the distance from the encoder to the true posterior. Maximizing the right side is minimizing

$$L = \underbrace{-\,\mathbb{E}_q[\log p(x \mid z, c)]}_{\text{reconstruction}} + \beta \cdot \mathrm{KL}\big(q \,\|\, p\big),$$

with $\beta = 1$ for the exact bound. We use $\beta_{\text{target}} = 0.1$ and raise β linearly from 0 over the first 20 epochs (KL annealing), so the decoder learns to use z before the KL term pulls the encoder towards the prior.

The reconstruction term is the masked loss of Section 2:

$$L_{\text{recon}} = \sum_k m_k \left[\ell_{\text{pos}}\big((u_k, v_k), (u_k^*, v_k^*)\big) + \lambda_{\text{rot}}\, \mathrm{CCE}(r_k)\right].$$

6.2 KL between a diagonal Gaussian and $\mathcal{N}(0, I)$

For one dimension, with $q = \mathcal{N}(\mu, \sigma^2)$ and $p = \mathcal{N}(0, 1)$:

$$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q(z) - \log p(z)\big] = \mathbb{E}_q\left[-\tfrac{1}{2}\log \sigma^2 - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}\right].$$

Using $\mathbb{E}_q[(z-\mu)^2] = \sigma^2$ and $\mathbb{E}_q[z^2] = \mu^2 + \sigma^2$:

$$\mathrm{KL} = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right).$$

The dimensions are independent, so the total is the sum over j. With $s_j = \log\sigma_j^2$ (the network's `logvar` output) this is the code's form $-\frac{1}{2}\sum_j(1 + s_j - \mu_j^2 - e^{s_j})$. Its gradients are

$$\frac{\partial\,\mathrm{KL}}{\partial\mu_j} = \mu_j, \qquad \frac{\partial\,\mathrm{KL}}{\partial s_j} = \tfrac{1}{2}\left(e^{s_j} - 1\right).$$

Both are zero exactly at $\mu_j = 0$, $\sigma_j = 1$, the prior.

6.3 Reparameterization

Sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ directly has no gradient with respect to μ and σ. Writing

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad \sigma = e^{s/2},$$

moves the randomness into ϵ, which does not depend on the parameters. Then

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial s} = \tfrac{1}{2}\,\sigma \odot \epsilon,$$

and the reconstruction gradient flows through the sample to the encoder.

6.4 Posterior collapse and active units

If the KL term dominates, the encoder outputs the prior for every input, the KL of every dimension is 0, and the decoder ignores z: all samples for a room look alike. We count *active units*: dimensions whose KL, averaged over the validation set, exceeds 0.01 nats. The default model has 8 of 16; the three frozen MAE models have 10, 9 and 10.

Used in: `kl_divergence`, `reparameterize`, `active_units`, `cvae_loss` in `spacegen/models/cvae.py`; annealing in `spacegen/train_cvae.py`. *Checked by:* tests that compare the KL with `torch.distributions`, check the reparameterization gradients, and prove that early stopping cannot fire during annealing; E5 for the effect of β.

---

7. Overlap: why area fails as a loss and penetration depth works

Take two boxes on one axis, with centres x_i, x_j and widths w_i, w_j.

Exact overlap width.

$$o_x = \mathrm{ReLU}\Big(\min\left(x_i + \tfrac{w_i}{2},\, x_j + \tfrac{w_j}{2}\right) - \max\left(x_i - \tfrac{w_i}{2},\, x_j - \tfrac{w_j}{2}\right)\Big).$$

If box j lies inside box i along x, both the $\min$ and the $\max$ pick box j's edges, so $o_x = w_j$ wherever j sits inside i, and $\partial o_x/\partial x_i = \partial o_x/\partial x_j = 0$. A contained item receives no gradient: the area cannot tell the optimizer which way to move it. The checker still uses the exact area (it is the right quantity to *measure*); the losses do not.

Penetration depth. With a clearance margin $\mu \geq 0$ and $d_x = x_i - x_j$:

$$p_x = \mathrm{ReLU}\Big(\frac{w_i + w_j}{2} + \mu - |d_x|\Big), \qquad \frac{\partial p_x}{\partial x_i} = -\,\mathrm{sign}(d_x) \;\text{ wherever } p_x > 0.$$

p_x is how far the boxes must slide apart along x to be separated by μ. Its gradient has size 1 and points away from the other box even under full containment. With p_y defined the same way,

$$\mathrm{pen}_{ij} = \min(p_x, p_y), \qquad L_{\mathrm{ov}} = \sum_{i<j} m_i\, m_j\, \mathrm{pen}_{ij}^2.$$

The $\min$ selects the axis along which the boxes separate fastest (the minimum translation), and the square makes the push proportional to the depth. Where the boxes are already apart, $\mathrm{pen} = 0$ and the gradient is 0: nothing to fix.

Smoothing. The code replaces $|d|$ by $\sqrt{d^2 + 10^{-6}}$, whose derivative $d/\sqrt{d^2 + 10^{-6}}$ is finite everywhere and exactly 0 when two centres coincide (by symmetry there is no preferred direction).

The same idea for the walls and the door. The protrusion out of the room is $\mathrm{out}_k = \sum_{\text{walls}} \mathrm{ReLU}(\text{distance past the wall})$, and the door term is the squared penetration depth between an item and the door's clearance zone.

Used in: `penetration_depth`, `overlap_penalty`, `out_of_room` in `spacegen/geometry.py`. *Checked by:* `test_contained_item_gets_a_push_that_the_exact_area_cannot_give`, the coincident-centre test, and finite-difference checks of the latent-optimization loss.

---

8. Latent optimization (M2)

A candidate $z_0 \sim \mathcal{N}(0, I)$ decodes to positions $p = \mathrm{dec}(z_0, c)$ that may break the hard rules. With the decoder's weights frozen, we minimize over z:

$$L_c(z) = \lambda_{\text{ov}} \sum_{i<j} \mathrm{pen}_{ij}^2 + \lambda_{\text{room}} \sum_k \mathrm{out}_k + \lambda_{\text{door}} \sum_k \mathrm{pen}_{k,\text{door}}^2 + \lambda_{\text{pin}} \sum_{k \in \text{pins}} \|p_k - p_k^{\text{pin}}\|^2 + \frac{\lambda_z}{2} \|z - z_0\|^2,$$

with $\lambda_{\text{ov}} = \lambda_{\text{room}} = \lambda_{\text{door}} = 10$, $\lambda_{\text{pin}} = 20$, $\lambda_z = 0.05$, a margin of 5 cm, Adam with learning rate 0.05 and at most 150 steps.

Gradient. By the chain rule, with $J = \partial p / \partial z$ the decoder's Jacobian:

$$\nabla_z L_c = J^\top \nabla_p L_{\text{geometry}} + \lambda_z (z - z_0).$$

The geometric gradient on the positions (Section 7) is carried back through the decoder into latent space. The items therefore move together along directions the decoder has learned, instead of each being pushed on its own.

The anchor. $\frac{\lambda_z}{2}\|z - z_0\|^2$ is a proximal (trust-region) term. Read as probabilities, the constraint terms act as a negative log-likelihood and the anchor as a Gaussian prior centred on the candidate's own start z_0. Shrinking $\|z\|^2$ instead would pull every candidate towards the same point $z = 0$ and destroy the diversity of the candidates.

Rotations are the arg-max of the decoder's logits at z_0 and stay fixed: an arg-max has no gradient.

Stopping. A candidate stops once its constraint terms (without the anchor) fall below 10^{-4}. A candidate that already satisfies them at z_0 keeps z_0.

Used in: `spacegen/latent_opt.py`. *Checked by:* finite-difference gradient checks (including a contained item), the test that M2 with 0 steps equals M1, and E8: raw validity rises with the number of steps while diversity is kept.

---

9. Metrics

$$\mathrm{RVR} = \frac{\#\,\text{raw samples passing H1 to H4}}{\#\,\text{raw samples}}, \qquad \text{Reachability} = \text{mean}\left(\frac{\text{reachable items}}{\text{items that need access}}\right).$$

Diversity of the valid layouts for one room, over the slots S present in both layouts of a pair:

$$\mathrm{Div} = \underset{\text{pairs } (a,b)}{\text{mean}} \; \frac{1}{|S|} \sum_{k \in S} \left(\|p_k^a - p_k^b\|_2 + 0.5\,[\,r_k^a \neq r_k^b\,] \right),$$

computed after canonicalization, so a symmetric item's meaningless flips do not count. The *diversity ratio* divides by the same quantity for the generator's layouts on the same rooms.

Classification (the evaluator): with true and false positives and negatives,

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \qquad \mathrm{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}.$$

Spearman correlation is the Pearson correlation of the ranks of the predicted and the rule scores: it measures whether the evaluator *orders* layouts like the rule score, which is what ranking needs.

Cost per valid layout $= (\text{time spent sampling, optimizing and checking}) / (\text{valid layouts produced})$.

Used in: `spacegen/metrics.py`, `spacegen/evaluate.py`, `spacegen/evaluator_report.py`.

---

10. The quality score and the hard checks

The rule score is a weighted sum of four terms, each in $[0, 1]$:

$$S = 0.30\, S_{\text{align}} + 0.30\, S_{\text{relations}} + 0.25\, S_{\text{circulation}} + 0.15\, S_{\text{space}}.$$

A relation (for example the sofa-to-TV distance) scores 1 inside its range $[a, b]$ and falls linearly to 0 at 0.5 m outside it:

$$\text{range_score}(t) = \max\!\left(0,\ 1 - \frac{\max(a - t,\ 0,\ t - b)}{0.5}\right).$$

The door's centre lies at $s = m_d + (L - 2m_d)\, o$ along a wall of length L, with $m_d = 0.65$ m and offset $o \in [0, 1]$. At $o = 0$ and $o = 1$ it is exactly m_d from a corner, so the 0.9 m door always fits.

The hard checks are H1 (every item inside the room), H2 (no pair overlaps by more than 0.005 m^2), H3 (the door's 0.9 × 0.9 m clearance zone is free) and H4 (every item that needs access can be reached from the door through cells at least 0.3 m from any obstacle).

Used in: `spacegen/quality.py`, `spacegen/rules.py`. Every threshold is our own assumption, listed in `configs/rules.yaml`.

---

11. What checks each derivation

Result	Check
Backpropagation, masked losses, softmax and Sigmoid gradients (Sections 2 and 4)	`notebooks/01_manual_backprop.ipynb`: hand gradients equal autograd to 1.8×10^{-15}
KL closed form (6.2)	test against `torch.distributions.kl_divergence` in `tests/test_cvae.py`
Reparameterization gradients (6.3)	test in `tests/test_cvae.py`
KL annealing and early stopping (6.1)	`test_early_stopping_cannot_fire_during_annealing`
Containment gradient of the penetration depth (7)	`test_contained_item_gets_a_push_that_the_exact_area_cannot_give` in `tests/test_geometry.py`
Latent-optimization gradient (8)	finite-difference checks in `tests/test_latent_opt.py`
M2 starts from M1's draws (8)	`test_m2_starts_from_m1s_draws` in `tests/test_headline.py`
Loss scale and effective β (2.3)	E2 and E5
Vanishing gradients, dying ReLU, BatchNorm (3)	E3b
Optimizers (5)	E4
Door placement (10)	`test_door_stays_on_its_wall_away_from_the_corners`